Univerzitet u Nišu, Elektronski fakultet
Katedra za računarstvo

Seminarski rad

Predikcija kardiovaskularnih bolesti korišćenjem metoda mašinskog učenja

Nemanja Kostić 1753

Predmet: **Veštačka inteligencija u medicini**

Predmetni profesor: **prof. dr Dragan S. Janković**

Niš, 2026

Sadržaj

1. UKRATKO O KARDIOVASKULARNIM BOLESTIMA.....2

1.1. UVOD..... 2

1.2 UZROCI I FAKTORI RIZIKA.....2

2. PREDIKCIJA BOLESTI.....3

3. KLASIFIKACIJA KVB-A UZ POMOĆ MAŠINSKOG UČENJA.....4

3.1. UVOD..... 4

3.2. KORIŠĆENI DATASET U PROJEKTU.....4

3.3. PREDPROCESIRANJE PODATAKA.....5

3.4. TRENIANJE MODELA.....8

3.5. UPOREĐIVANJE REZULTATA.....18

3.6. *HYPERPARAMETERS TUNING* ALGORITAMA.....18

4. ZAKLJUČAK.....20

5. LITERATURA.....21

1. Ukratko o kardiovaskularnim bolestima

1.1. Uvod

Kardiovaskularne bolesti predstavljaju grupu oboljenja srca i krvnih sudova koja uključuje koronarnu bolest srca, moždani udar, srčanu insuficijenciju, hipertenziju i perifernu arterijsku bolest. Prema podacima Svetske zdravstvene organizacije, od ovih bolesti svake godine umire oko 17,9 miliona ljudi – gotovo 32% svih globalnih smrtnih slučajeva.

U Srbiji, kardiovaskularne bolesti su odgovorne za više od polovine svih smrtnih slučajeva, što ih svrstava u apsolutni prioritet javnog zdravlja.

1.2 Uzroci i faktori rizika

Razvoj kardiovaskularnih bolesti najčešće je rezultat dugotrajnog delovanja više faktora rizika istovremeno. Oni se dele na nepromenljive – poput genetske predispozicije, starosti i pola – i promenljive, na koje je moguće uticati.

Ključni promenljivi faktori su:

- **arterijska hipertenzija**, opterećuje zidove krvnih sudova i srčani mišić
- **dislipidemija**, odnosno povišen nivo LDL holesterola koji ubrzava proces ateroskleroze
- **dijabetes mellitus**, koji oštećuje krvne sudove hroničnim povišenjem šećera u krvi
- **pušenje**, koje direktno oštećuje endotel krvnih sudova i podstiče formiranje krvnih ugrušaka
- **gojaznost i fizička neaktivnost**, koje pogoršavaju sve prethodno navedene faktore
- **hronični stres**, česta izloženost stresu.
- **nezdrava ishrana**, u glavnom bogata zasićenim mastima i solju.

Ateroskleroza – postepeno sužavanje arterija usled taloženja masnih naslaga – zajednički je mehanizam koji je u osnovi većine kardiovaskularnih incidenata. Kada plak (naslaga koja se postepeno formira na unutrašnjim zidovima arterija. Sastoji se uglavnom od masnoća, posebno LDL holesterola, kalcijuma, upalnih ćelija i vezivnog tkiva) u koronarnoj arteriji naglo pukne, dolazi do tromboze i akutnog infarkta miokarda. Akutni infarkt miokarda je ono što se popularno zove "srčani udar". Nastaje kada se dotok krvi u deo srčanog mišića potpuno prekine, srčani mišić je specifičan po tome što stalno radi i što mu je potreban neprekidan dotok kiseonika. Kada arterija bude začepljena, ćelije tog dela mišića počinju da odumiru – i to relativno brzo, već nakon 20-40 minuta bez kiseonika. Što duže začepljenje traje, veće je oštećenje.

2. Predikcija bolesti

Moderna medicina raspolaže preciznim alatima za procenu individualnog kardiovaskularnog rizika pre nego što se bolest uopšte razvije. Najšire korišćen je SCORE sistem (Systematic COronary Risk Evaluation), koji je razvilo Evropsko kardiološko društvo. SCORE algoritam na osnovu nekoliko parametara – starosti, pola, statusa pušača, vrednosti sistolnog krvnog pritiska i ukupnog holesterola – izračunava desetogodišnju verovatnoću fatalnog kardiovaskularnog događaja. Rezultat se izražava u procentima i svrstava pacijenta u kategoriju niskog, umerenog, visokog ili veoma visokog rizika, što direktno određuje agresivnost lečenja. SCORE *kalkulatori* su dostupni na internetu (https://www.mdcalc.com/calc/10499/systematic-coronary-risk-evaluation-score2).

Pored SCORE-a, u upotrebi su i Framinghamska skala rizika, razvijena na osnovu decenijskog praćenja populacije u SAD-u, kao i noviji SCORE2 model koji uključuje i rizik od nefatalnih događaja. U kliničkoj praksi se sve češće kombinuju i biomarkeri iz krvi – poput hsCRP-a (visokosenzitivni C-reaktivni protein) i NT-proBNP-a – koji pružaju dodatne informacije o upalnim procesima i opterećenju srca. Savremeni pristup ide i korak dalje: kalcijumski skor koronarnih arterija, dobijen CT snimanjem, direktno meri stepen ateroskleroze i može da promeni procenu rizika čak i kod pacijenata koji prema klasičnim skalama spadaju u umerenu kategoriju.

Pored tradicionalnih kliničkih alata poput SCORE sistema, savremena medicina sve više okreće pogled ka metodama veštačke inteligencije i mašinskog učenja kao novoj generaciji alata za predikciju kardiovaskularnih rizika. Za razliku od klasičnih skala koje se oslanjaju na svega nekoliko unapred odabranih parametara, algoritmi mašinskog učenja imaju mogućnost obrade stotine varijabli (*feature*-a): od laboratorijskih nalaza i genetskih markera, preko podataka o životnom stilu, do vrednosti dobijenih nošenjem pametnih uređaja (npr. smart watch-a). Ključna prednost ovog pristupa leži u sposobnosti otkrivanja složenih, nelinearnih obrazaca u podacima koji su nedostupni tradicionalnim statističkim modelima. Studije su pokazale da modeli mašinskog učenja trenirani na velikim kliničkim bazama podataka mogu da nadmaše konvencionalne skale rizika u tačnosti predikcije, posebno kod pacijenata sa atipičnim profilima rizika koje klasični alati nedovoljno prepoznaju. Pored toga, ovi modeli imaju potencijal da se kontinuirano unapređuju kako je postojanje novih podataka sve veće.

U ovom radu, pokušaćemo da uvidimo neke metode i benefite korišćenja mašinskog učenja u ovoj oblasti.

3. Klasifikacija KVB-a uz pomoć mašinskog učenja

3.1. Uvod

U ovom radu istražujemo primenu metoda mašinskog učenja za predikciju kardiovaskularnih bolesti kao problem binarne klasifikacikacije, gde model na osnovu skupa podataka o pacijentima predviđa rizik bolesti. Korišćeno je pet algoritama: *Logistic regresion*, *Naive Bayes*, *K-nearest neighbours, Random Forest* i *Gradient Boosting*. Poređenjem performansi ovih algoritama cilj je bio utvrditi koji pristup najefikasnije i najpouzdanije identifikuje pacijente pod rizikom, uz analizu najvažnijih faktora koji doprinose predikciji.

3.2. Korišćeni dataset u projektu

U ovom projektu korišćen je skup podataka dostupan na platformi *Kaggle (https://www.kaggle.com/datasets/bertnardomariouskono/cardiovascular-disease-risk-prediction-dataset/data)*, koji sadrži 15.000 sintetički generisanih zapisa o zdravstvenim parametrima

pacijenata, namenjenih predviđanju rizika od kardiovaskularnih bolesti. Skup obuhvata širok spektar *feature*-a – od demografskih podataka i vitalnih znakova, preko laboratorijskih nalaza poput nivoa holesterola i šećera u krvi, do faktora životnog stila kao što su pušenje, fizička aktivnost i kvalitet sna. *Feature* nad kojim ćemo vršiti predikciju je binaran: vrednost 0 označava nizak rizik, a vrednost 1 visok rizik od kardiovaskularne bolesti. Podaci su generisani u Python-u korišćenjem biblioteka NumPy i Pandas, pri čemu su labele kreirane na osnovu izračunatog skora rizika inspirisanog Framinghamskim principima.

Upotreba sintetičkih podataka u medicinskim istraživanjima nosi niz značajnih prednosti. Najvažnija od njih je zaobilaženje etičkih i pravnih ograničenja vezanih za privatnost pacijenata – realni medicinski podaci su strogo zaštićeni i teško dostupni istraživačima, posebno u akademskom kontekstu. Sintetički podaci, generisani prema kliničkim heuristikama, omogućavaju reprodukciju realističnih korelacija između medicinskih varijabli bez izlaganja bilo čijih ličnih podataka riziku. Pored toga, ovakvi skupovi podataka ne pate od problema nepotpunih ili nedostajućih vrednosti koji su česti u realnim kliničkim bazama, a moguće ih je kreirati u željenom obimu, što olakšava treniranje i evaluaciju modela mašinskog učenja. Iako sintetički podaci ne mogu u potpunostizameniti realne kliničke skupove u finalnoj validaciji medicinskih sistema, oni

predstavljaju izuzetno vredan alat u ranim fazama razvoja i testiranja algoritama za predikciju bolesti.

	Patient_ID	Age	Gender	Height_cm	Weight_kg	BMI	Systolic_BP	Diastolic_BP	Cholesterol_Total	Cholesterol_LDL	Cholesterol_HDL	Fasting_Blood_Sugar	Smoking_Status	Alcohol_Consumption
0	PID-00001	60	0	146.9	51.3	23.8	140	89	217	151	52	83	0	1
1	PID-00002	53	0	161.8	76.6	29.3	128	81	203	119	38	116	0	0
2	PID-00003	62	1	174.7	92.4	30.3	141	100	173	124	45	90	0	0
3	PID-00004	73	1	173.3	68.9	22.9	136	96	193	117	45	81	0	0
4	PID-00005	52	1	178.6	79.8	25.0	122	80	236	153	41	79	0	1
5	PID-00006	52	0	159.6	60.3	23.7	134	92	225	155	48	103	0	0
6	PID-00007	73	0	158.2	62.7	25.0	149	98	260	166	46	103	0	0
7	PID-00008	64	0	144.5	72.0	34.5	129	90	237	160	44	108	0	1
8	PID-00009	49	0	168.7	80.2	28.2	139	91	227	138	37	90	1	0
9	PID-00010	61	1	176.8	113.6	36.3	131	91	206	139	38	103	0	0
10	PID-00011	49	0	157.4	83.9	33.9	150	100	188	129	43	106	1	0
11	PID-00012	49	0	162.2	84.5	32.1	141	91	239	149	38	122	0	1
12	PID-00013	57	0	158.7	84.9	33.7	126	90	231	152	42	99	0	1
13	PID-00014	32	0	161.2	83.8	32.3	137	92	204	135	48	104	1	1
14	PID-00015	34	1	179.6	112.9	35.0	124	89	199	127	34	124	0	0

Izgled dataset-a

3.3. Predprocesiranje podataka

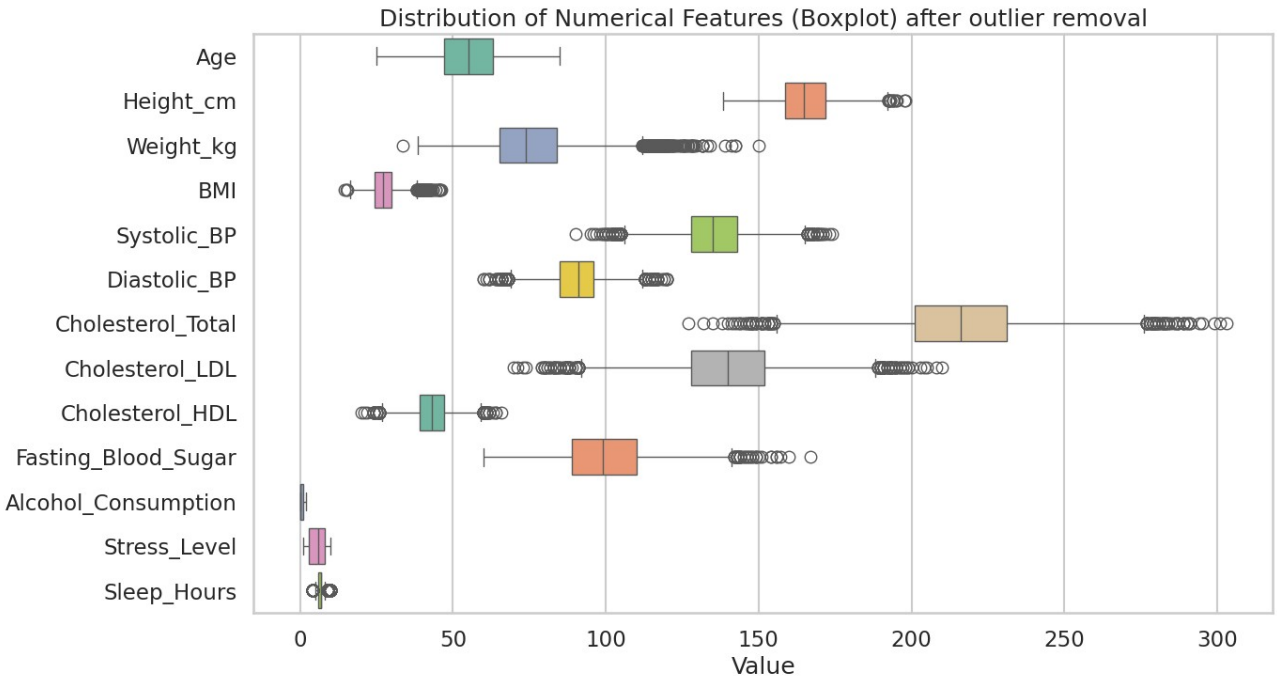
Ovaj korak je deo pripreme podataka pre treniranja modela kako bi model bio precizniji i nekontaminiran lošim podacima.

```
# Brisanje podataka sa null vrednostima, augmentacija u ovom datasetu nije potrebna
dataframe.dropna(inplace=True)

# ID pacijenta nam nije potreban
dataframe = dataframe.drop("Patient_ID", axis=1)
```

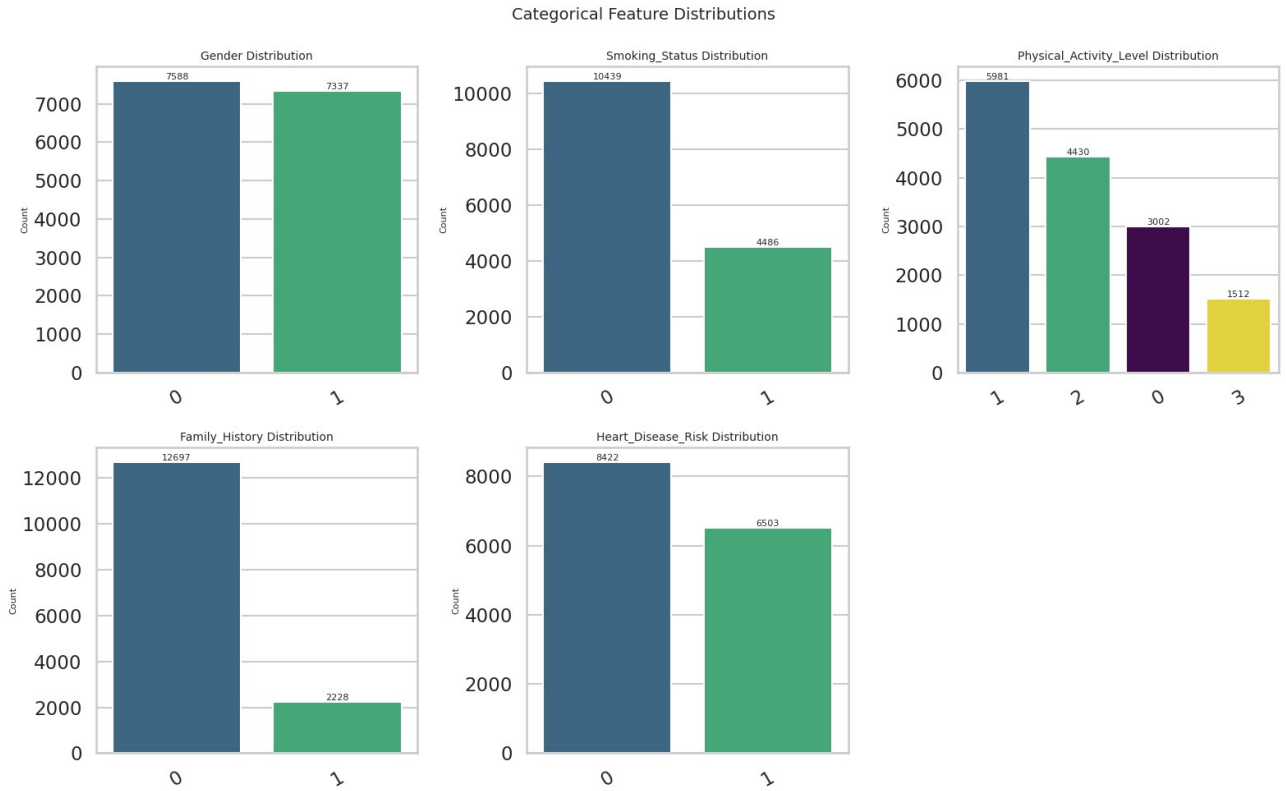
`dropna()` uklanja sve redove u kojima postoji bar jedna nedostajuća vrednost. Parametar `inplace=True` znači da se izmena primenjuje direktno na postojeći dataframe, umesto da se kreira novi. U ovom datasetu augmentacija tj. popunjavanje nedostajućih vrednosti procenjenim vrednostima (npr. srednjom vrednošću kolone) nije bila potrebna jer sintetički generisani podaci su već potpuni i bez praznina, pa je ovo uglavnom preventivna mera. `drop("Patient_ID", axis=1)` uklanja kolonu sa identifikatorom pacijenta. Parametar axis=1 govori pandas-u da briše kolonu (a ne red). `Patient_ID` se uklanja jer je to proizvoljan redni broj koji ne nosi nikakvu medicinsku informaciju.

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

iso = IsolationForest(
    n_estimators=300,
    contamination=0.005,
    random_state=42
)

outliers = iso.fit_predict(X_scaled)
dataframe = dataframe[outliers == 1]
```

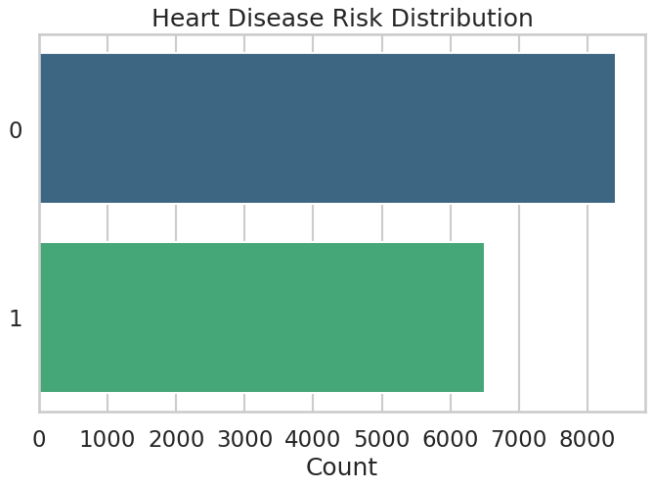
IsolationForest je algoritam koji identifikuje anomalije u podacima. Logika mu je intuitivna: normalne tačke je "teže izolovati" jer liče na ostatak podataka, dok se ekstremne vrednosti brzo odvajaju. Parametar contamination=0.005 govori modelu da očekuje oko 0.5%



outliera u datasetu, a n_estimators=300 definiše broj stabala odluke koja se grade. Nakon što model označi svaki red kao normalan (1) ili outlier (-1), linija dataframe[outliers == 1] zadržava samo normalne zapise i izbacuje anomalije. Takođe, s obzirom da je ovo sintetički *dataset*, u većini slučajeva nema pojavljivanja *outlier*-a



Distribucija kategoričkih vrednosti



Po prikazanim graficima, možemo zaključiti da je *dataset* poprilično izbalansiran.

```
X = dataframe.drop('Heart_Disease_Risk', axis=1)
y = dataframe['Heart_Disease_Risk']

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    stratify=y,
    random_state=42
)
```

`train_test_split` deli podatke na trening i test skup. Parametar test_size=0.2 znači da 80% podataka ide u trening, a 20% u test podatke. Trening skup služi za treniranje modela, a test skup simulira "neviđene" podatke i koristi se isključivo za finalnu evaluaciju.

`stratify=y` je važan parametar koji osigurava da raspored klasa (0 i 1) bude proporcionalno jednak i u trening i u test skupu. Na primer, ako u celom datasetu ima 60% nizkog rizika i 40% visokog, isti taj odnos će biti očuvan u oba skupa. Bez ovoga, moglo bi se desiti da test skup slučajno sadrži previše jedne klase, što bi dalo nerealne rezultate evaluacije.

`random_state=42` osigurava reproduktivnost – svaki put kada se kod pokrene, podela će biti identična.

3.4. Trenianje modela

Korišćeno je pet algoritama: *Logistic regresion*, *Naive Bayes*, *K-nearest neighbours, Random Forest* i *Gradient Boosting*.

U daljim primenama ovih algoritama, biće korišćene neke funkcije kako bi smanjile ponavljanje koda (*boiler plate*)

```
class ModelResult:
    def __init__(self, name, cv_metrics, y_pred_test=None, y_proba_test=None):
        self.name = name
        self.cv_metrics = cv_metrics
        self.y_pred_test = y_pred_test
        self.y_proba_test = y_proba_test

results_list = []  # Rezultati svakog algoritma
tuned_parameters_results_list = [] # Rezultati algoritama posle fine-tuning-a hyper
parametara

def evaluate_cv(pipeline, X_train, y_train, cv):
    scores = cross_validate(
        pipeline,
        X_train,
        y_train,
        cv=cv,
        scoring={
            'accuracy': 'accuracy',
            'macro_f1': 'f1_macro',
            'macro_recall': 'recall_macro'
        }
    )

    return {
        "accuracy": scores['test_accuracy'].mean(),
        "macro_f1": scores['test_macro_f1'].mean(),
        "macro_recall": scores['test_macro_recall'].mean()
    }

def train_and_predict(pipeline, X_train, y_train, X_test):
    pipeline.fit(X_train, y_train)
    y_pred = pipeline.predict(X_test)
    y_proba = pipeline.predict_proba(X_test)
    return y_pred, y_proba

def plot_confusion_matrix(y_true, y_pred, labels, title):
    cm = confusion_matrix(y_true, y_pred)
    disp = ConfusionMatrixDisplay(cm, display_labels=labels)

    plt.figure(figsize=(8, 6))
    disp.plot(cmap="Blues", values_format="d")
```

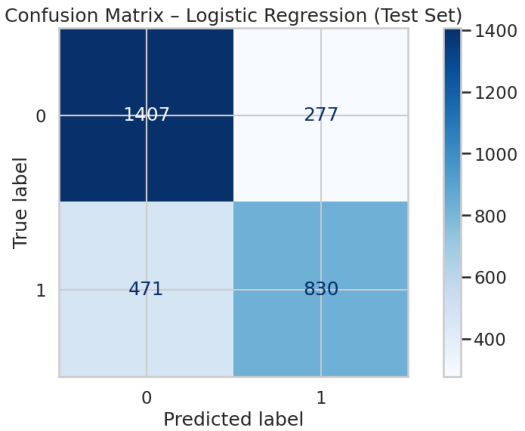
```
    plt.title(title)
    plt.show()

def plot_roc(y_true, y_proba, classes, title):
    plt.figure()

    if len(classes) == 2:
        proba = y_proba[:, 1] if y_proba.ndim > 1 else y_proba
        fpr, tpr, _ = roc_curve(y_true, proba)
        roc_auc = auc(fpr, tpr)
        plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.3f}")
    else:
        y_bin = label_binarize(y_true, classes=classes)
        roc_auc = {}
        for i, cls in enumerate(classes):
            fpr, tpr, _ = roc_curve(y_bin[:, i], y_proba[:, i])
            roc_auc[cls] = auc(fpr, tpr)
            plt.plot(fpr, tpr, label=f"Class {cls} AUC = {roc_auc[cls]:.3f}")

    plt.plot([0, 1], [0, 1], '--', color='gray')
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    plt.title(title)
    plt.legend()
    plt.tight_layout()
    plt.show()

    return roc_auc

def plot_cv_metrics(results_list, metrics=["accuracy","macro_f1","macro_recall"]):

    data = []
    for res in results_list:
        row = {"Algorithm": res.name}
        for m in metrics:
            row[m] = res.cv_metrics[m]
        data.append(row)

    df = pd.DataFrame(data)
    df_melted = df.melt(id_vars="Algorithm", var_name="Metric", value_name="Score")

    plt.figure(figsize=(10,6))
    ax = sn.barplot(data=df_melted, x="Algorithm", y="Score", hue="Metric")

    plt.ylim(0, 1.05)
    plt.title("CV Metrics Comparison")
    plt.xticks(rotation=45)
    plt.ylabel("Score")
    plt.legend(loc="lower right")

    for p in ax.patches:
        height = p.get_height()
        x = p.get_x() + p.get_width() / 2

        if height > 0.15:
            va = "top"
            y = height - 0.02
            color = "white"
            weight = "bold"
        else:
            va = "bottom"
            y = height + 0.02
            color = "black"
            weight = "normal"

        ax.text(x, y, f"{height:.3f}", ha="center", va=va, color=color, fontsize=9,
fontweight=weight)
```

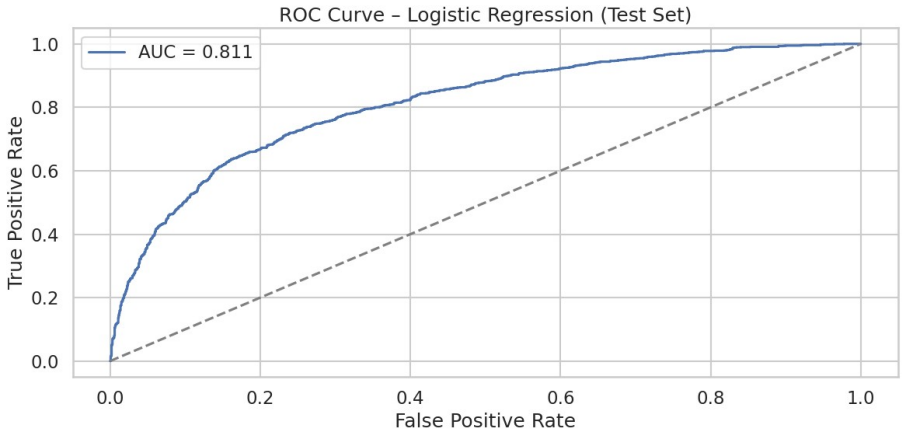
```
    plt.tight_layout()
    plt.show()

def plot_feature_importance(model_pipeline, feature_names, top_n=None, title=None):
    if hasattr(model_pipeline, "named_steps"):
        steps = list(model_pipeline.named_steps.keys())
        model = model_pipeline.named_steps[steps[-1]]

        for step_name in steps[:-1]:
            step = model_pipeline.named_steps[step_name]
            if hasattr(step, 'get_feature_names_out'):
                try:
                    feature_names = list(step.get_feature_names_out())
                except Exception:
                    pass
    else:
        model = model_pipeline

    if hasattr(model, "feature_importances_"):
        importances = model.feature_importances_
    elif hasattr(model, "coef_"):
        importances = np.mean(np.abs(model.coef_), axis=0) if model.coef_.ndim > 1 else
np.abs(model.coef_[0])

    if len(feature_names) != len(importances):
        feature_names = [f"feature_{i}" for i in range(len(importances))]

    importance_df = pd.DataFrame({
        "feature": feature_names,
        "importance": importances
    }).sort_values(by="importance", ascending=False)

    if top_n:
        importance_df = importance_df.head(top_n)

    plt.figure()
    plt.barh(importance_df["feature"], importance_df["importance"])
    plt.gca().invert_yaxis()
    plt.title(title if title else "Feature Importance")
    plt.xlabel("Importance")
    plt.tight_layout()
    plt.show()

    return importance_df

def plot_correlation_matrix(X, numerical_features, title="Feature Correlation Matrix"):
    corr = X[numerical_features].corr()

    mask = np.triu(np.ones_like(corr, dtype=bool), k=1)

    plt.figure(figsize=(14, 10))
    sn.heatmap(
        corr,
        mask=mask,
        annot=True,
        fmt=".1f",
        cmap="coolwarm",
        center=0,
        square=True,
        linewidths=0.5,
        annot_kws={"size": 8}
    )
    plt.title(title, fontsize=14)
    plt.xticks(rotation=45, ha='right', fontsize=9)
    plt.yticks(fontsize=9)
    plt.tight_layout()
    plt.show()
```

3.4.1. *Logistic regresion* algoritam

Logistička regresija računa linearnu kombinaciju ulaznih promenljivih i propušta je kroz sigmoidnu funkciju, koja taj rezultat pretvara u verovatnoću između 0 i 1. Ako je verovatnoća iznad 0.5, pacijent se klasifikuje u kategoriju visokog rizika, u suprotnom kao nizak. Zbog svoje jednostavnosti i interpretabilnosti često se koristi kao baseline model sa kojim se porede složeniji algoritmi.

```
pipeline_rf = Pipeline([
    ('scaler', StandardScaler()),
    ('model', LogisticRegression())
])

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

cv_results = evaluate_cv(pipeline_rf, X_train, y_train, cv)

print("Logistic Regression (CV mean):")
for k, v in cv_results.items():
    print(f"{k.capitalize():15}: {v:.3f}")

y_pred_test, y_proba_test = train_and_predict(
    pipeline_rf, X_train, y_train, X_test
)

classes = np.unique(y_train)

plot_confusion_matrix(
    y_test, y_pred_test, classes,
    "Confusion Matrix – Logistic Regression (Test Set)"
)

plot_roc(
    y_test, y_proba_test, classes,
    "ROC Curve – Logistic Regression (Test Set)"
)

plot_feature_importance(pipeline_rf, feature_names, top_n=15, title="Feature Importance –
Logistic Regression")

rf_result = ModelResult(
    name="Logistic Regression",
    cv_metrics=cv_results,
    y_pred_test=y_pred_test,
    y_proba_test=y_proba_test
)

results_list.append(rf_result)
```



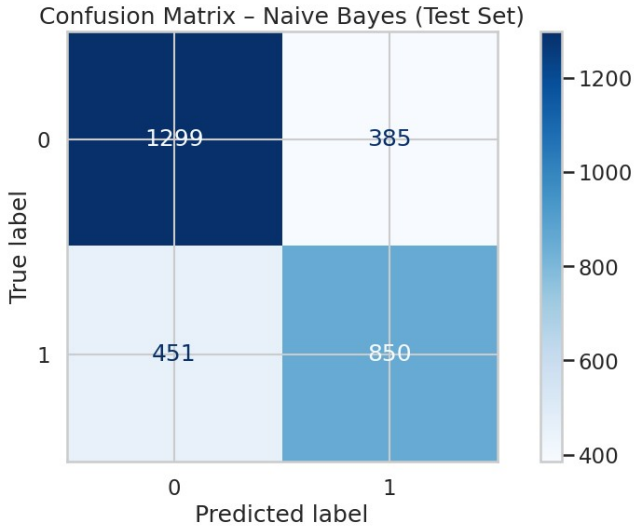


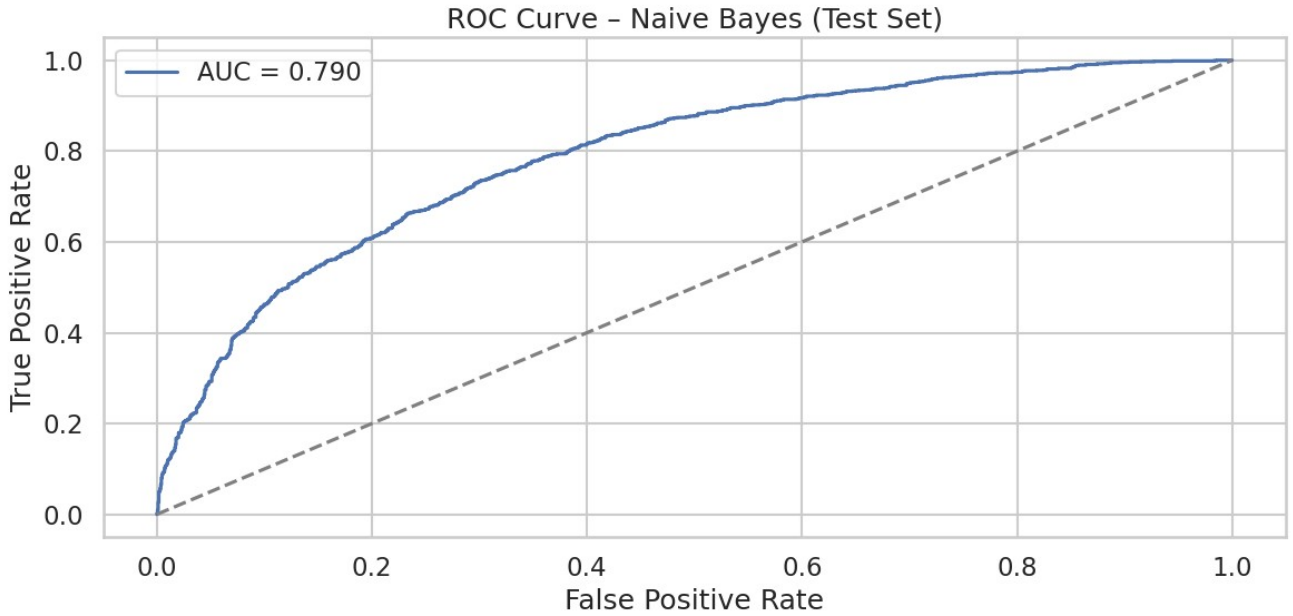
3.4.2. *Naive Bayes* algoritam

Naive Bayes se zasniva na Bayesovoj teoremi, za svakog pacijenta računa verovatnoću da pripada određenoj klasi na osnovu ulaznih promenljivih. "Naivni" deo dolazi od pretpostavke da su sve promenljive međusobno nezavisne, što u medicinskim podacima retko važi (npr. BMI i krvni pritisak su povezani), ali algoritam i pored te pretpostavke u praksi daje solidne rezultate. Brz je i efikasan, ali zbog te naivne pretpostavke generalno slabiji od složenijih modela.

```
pipeline_rf = Pipeline([
    ('scaler', StandardScaler()),
    ('model', GaussianNB())
])

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

cv_results = evaluate_cv(pipeline_rf, X_train, y_train, cv)

print("Naive Bayes (CV mean):")
for k, v in cv_results.items():
    print(f"{k.capitalize():15}: {v:.3f}")

y_pred_test, y_proba_test = train_and_predict(
    pipeline_rf, X_train, y_train, X_test
)

classes = np.unique(y_train)

plot_confusion_matrix(
    y_test, y_pred_test, classes,
    "Confusion Matrix – Naive Bayes (Test Set)"
)

plot_roc(
    y_test, y_proba_test, classes,
    "ROC Curve – Naive Bayes (Test Set)"
)


rf_result = ModelResult(
    name="Naive Bayes",
    cv_metrics=cv_results,
    y_pred_test=y_pred_test,
    y_proba_test=y_proba_test
)
```

```
results_list.append(rf_result)
```



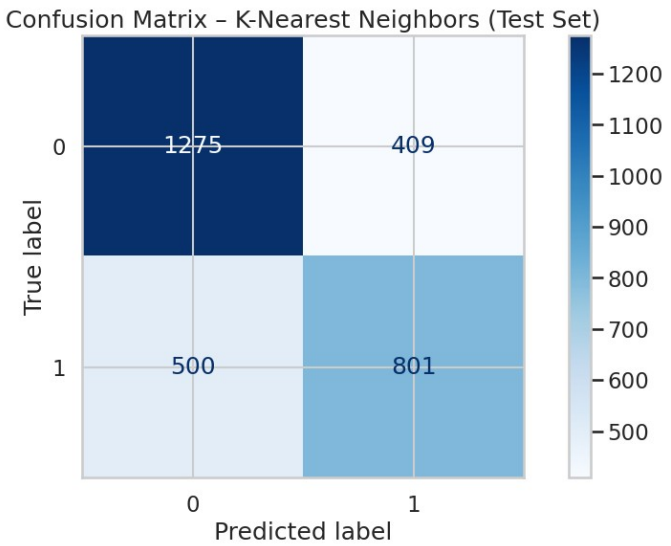


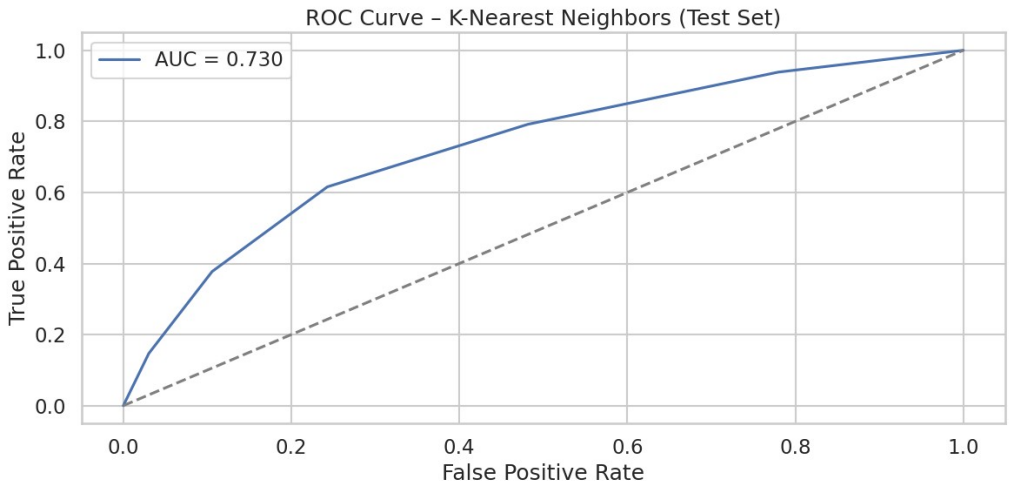
3.4.3. *K-nearest neighbours* algoritam

KNN klasifikuje pacijenta tako što pronađe K najsličnijih pacijenata u trening skupu i dodeli mu klasu koja je najzastupljenija među njima, jednostavno glasanje. Sličnost se meri euklidskom distancom u prostoru *feature*-a, što je i razlog zašto je skaliranje podataka pre primene ovog algoritma neophodno. Glavni nedostatak je što je spor na velikim skupovima podataka jer za svaku predikciju mora da poredi novi primer sa svim trening primerima.

```
pipeline_rf = Pipeline([
    ('scaler', StandardScaler()),
    ('model', KNeighborsClassifier())
])

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

cv_results = evaluate_cv(pipeline_rf, X_train, y_train, cv)
```

```
print("K-Nearest Neighbors (CV mean):")
for k, v in cv_results.items():
    print(f"{k.capitalize():15}: {v:.3f}")

y_pred_test, y_proba_test = train_and_predict(
    pipeline_rf, X_train, y_train, X_test
)

classes = np.unique(y_train)

plot_confusion_matrix(
    y_test, y_pred_test, classes,
    "Confusion Matrix – K-Nearest Neighbors (Test Set)"
)

plot_roc(
    y_test, y_proba_test, classes,
    "ROC Curve – K-Nearest Neighbors (Test Set)"
)


rf_result = ModelResult(
    name="K-Nearest Neighbors",
    cv_metrics=cv_results,
    y_pred_test=y_pred_test,
    y_proba_test=y_proba_test
)

results_list.append(rf_result)
```

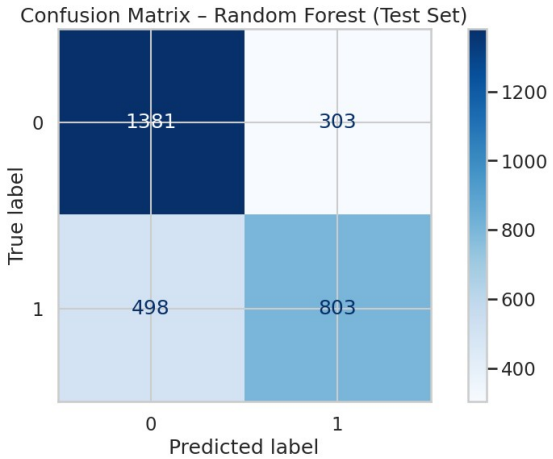


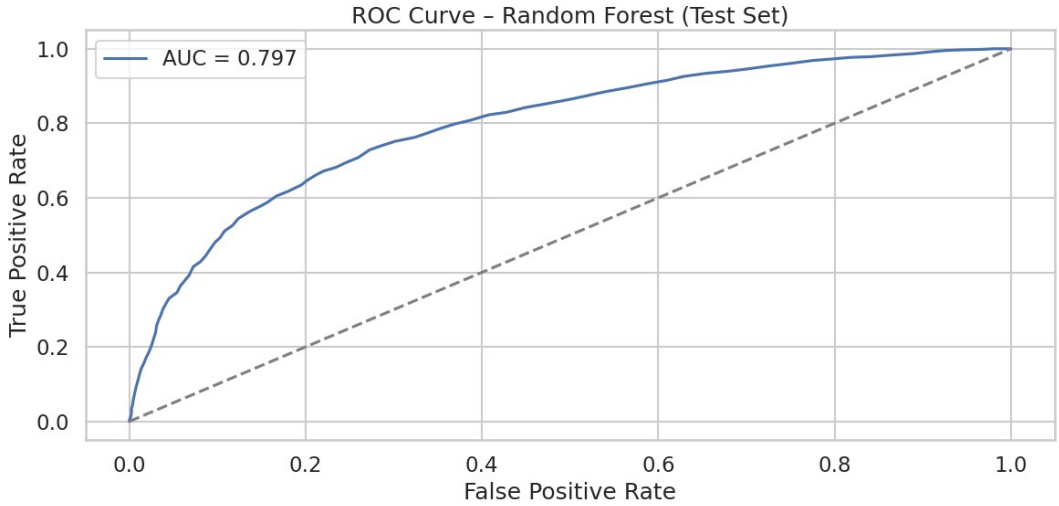


3.4.4. *Random Forest* algoritam

Random Forest gradi veliki broj stabala odluke, pri čemu svako stablo uči na nasumičnom podskupu podataka i promenljivih. Finalna predikcija se donosi glasanjem – klasa koju je predvidela većina stabala postaje krajnji rezultat. Ovaj pristup značajno smanjuje *overfitting* u poređenju sa pojedinačnim stablom odluke, jer greške pojedinih stabala se međusobno poništavaju. Spada u jedan od najpouzdanijih algoritama opšte namene i kao dodatnu vrednost pruža uvid u važnost pojedinih promenljivih, što je korisno u medicinskom kontekstu.

```
pipeline_rf = Pipeline([
    ('scaler', StandardScaler()),
    ('model', RandomForestClassifier())
])

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

cv_results = evaluate_cv(pipeline_rf, X_train, y_train, cv)

print("Random Forest (CV mean):")
for k, v in cv_results.items():
    print(f"{k.capitalize():15}: {v:.3f}")

y_pred_test, y_proba_test = train_and_predict(
    pipeline_rf, X_train, y_train, X_test
)

classes = np.unique(y_train)

plot_confusion_matrix(
    y_test, y_pred_test, classes,
    "Confusion Matrix – Random Forest (Test Set)"
)

plot_roc(
    y_test, y_proba_test, classes,
    "ROC Curve – Random Forest (Test Set)"
)

plot_feature_importance(pipeline_rf, feature_names, top_n=15, title="Feature Importance –
Random Forest")

rf_result = ModelResult(
    name="Random Forest",
    cv_metrics=cv_results,
    y_pred_test=y_pred_test,
    y_proba_test=y_proba_test
)

results_list.append(rf_result)
```





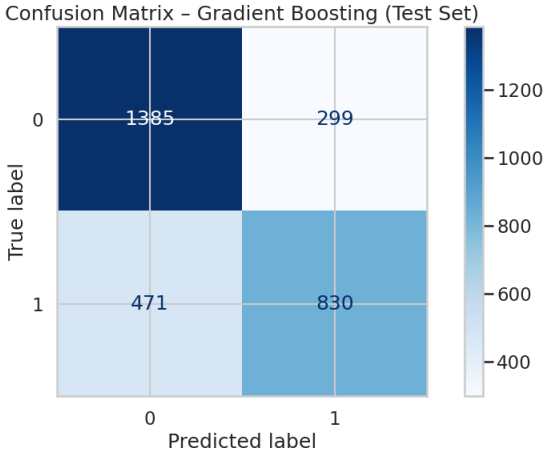
3.4.5. *Gradient Boosting* algoritam

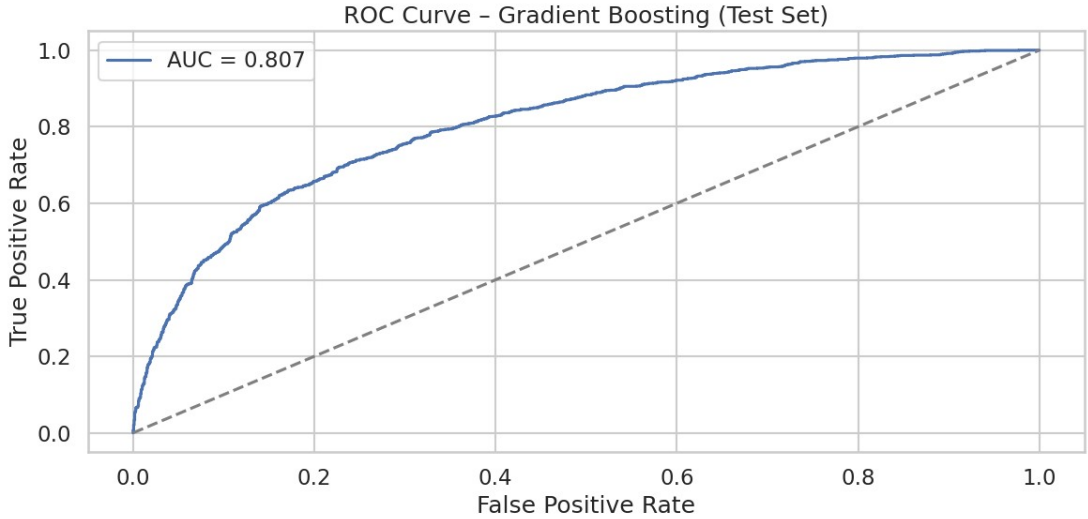
Gradient Boosting, za razliku od Random Foresta, gradi stabla sekvencijalno – svako novo stablo uči na greškama prethodnog i pokušava da ih ispravi. Na taj način model se iterativno poboljšava, fokusirajući se na primere koje je teško klasifikovati. Rezultat je tipično vrlo precizan model, ali zbog sekvencijalne prirode sporiji je za treniranje od Random Foresta i osetljiviji na izbor hiperparametara.

")

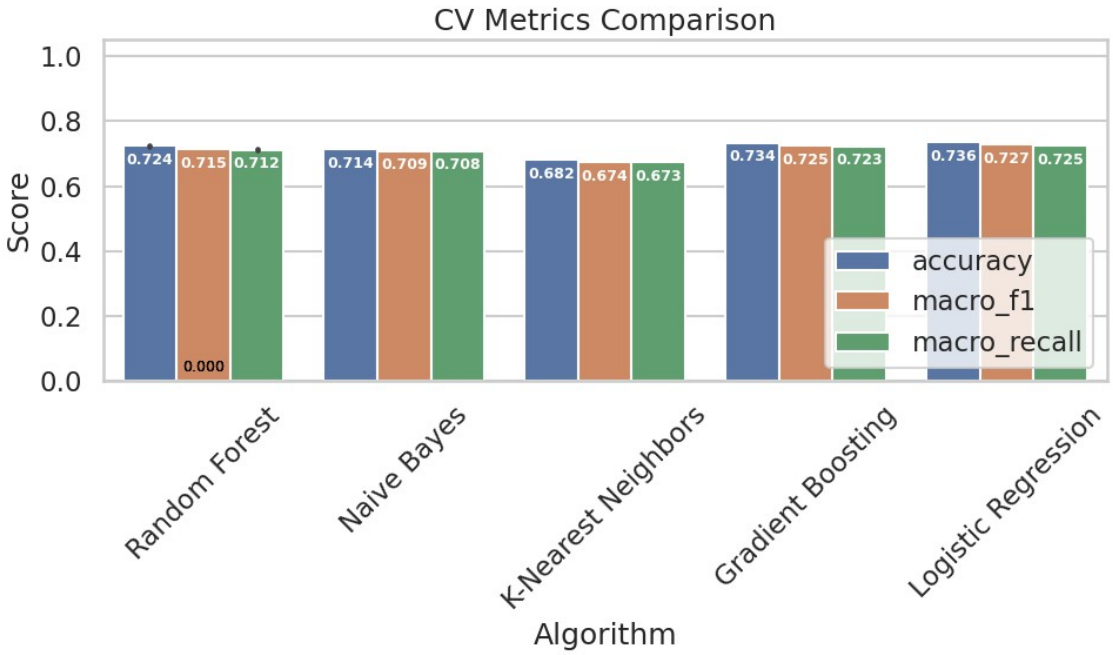
```
pipeline_rf = Pipeline([
    ('scaler', StandardScaler()),
    ('model', GradientBoostingClassifier())
])

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

cv_results = evaluate_cv(pipeline_rf, X_train, y_train, cv)

print("Gradient Boosting (CV mean):")
for k, v in cv_results.items():
    print(f"{k.capitalize():15}: {v:.3f}")

y_pred_test, y_proba_test = train_and_predict(
```

```
    pipeline_rf, X_train, y_train, X_test
)

classes = np.unique(y_train)

plot_confusion_matrix(
    y_test, y_pred_test, classes,
    "Confusion Matrix – Gradient Boosting (Test Set)"
)

plot_roc(
    y_test, y_proba_test, classes,
    "ROC Curve – Gradient Boosting (Test Set)"
)

plot_feature_importance(pipeline_rf, feature_names, top_n=15, title="Feature Importance –
Gradient Boosting")

rf_result = ModelResult(
    name="Gradient Boosting",
    cv_metrics=cv_results,
    y_pred_test=y_pred_test,
    y_proba_test=y_proba_test
)

results_list.append(rf_result)
```





3.5. Upoređivanje rezultata



Metrike koje su korišćene,

- *Accuracy* – procenat tačno klasifikovanih pacijenata
- *Macro F1* – harmonijska sredina preciznosti i *recall*-a, izračunata jednako za obe klase
- *Macro Recall* – sposobnost modela da pronađe sve stvarne pozitivne slučajeve, takođe ujednačena po klasama

Što se tiče rezultata, *Gradient Boosting* i *Logistic regresion* su se pokazali kao najbolji modeli sa vrlo sličnim skorovima (*accuracy* ~0.735), dok *KNN* zaostaje sa najnižim vrednostima across svih metrika. Generalno, vrednosti oko 0.72–0.73 su solidne za sintetički dataset, ali u medicinskom kontekstu bi bilo poželjno koristiti *recall* za klasu visokog rizika, jer je propuštanje bolesnog pacijenta skuplja greška od lažnog alarma u našem slučaju. Treba napomenuti da se ne treba uvek voditi *recall* metrikom u medicinskim predikcijama stanja. Šta ako na primer imamo klasifikaciju za neku neobaveznu hiruršku operaciju? U tom slučaju bi bilo pametnije koristiti *precision* metriku.

3.6. *Hyperparameters tuning* algoritama

Nakon inicijalne evaluacije modela putem *cross* validacije, sledeći korak je optimizacija hiperparametara za dva najbolje rangirana algoritma, *Gradient Boosting* i *Logistic regresion*. Za pretragu optimalnih vrednosti korišćena je metoda *GridSearchCV*, koja proverava sve kombinacije zadatih hiperparametara i bira one koje daju najbolje performanse na osnovu *cross* validacije.

Kod Logističke regresije podešavani su parametar regularizacije C, koji kontroliše kompromis između prilagođavanja trening podacima i generalizacije, te tip regularizacije (L1 i L2). Manje vrednosti C znače jaču regularizaciju i jednostavniji model, dok veće vrednosti dozvoljavaju modelu da se više prilagodi podacima.

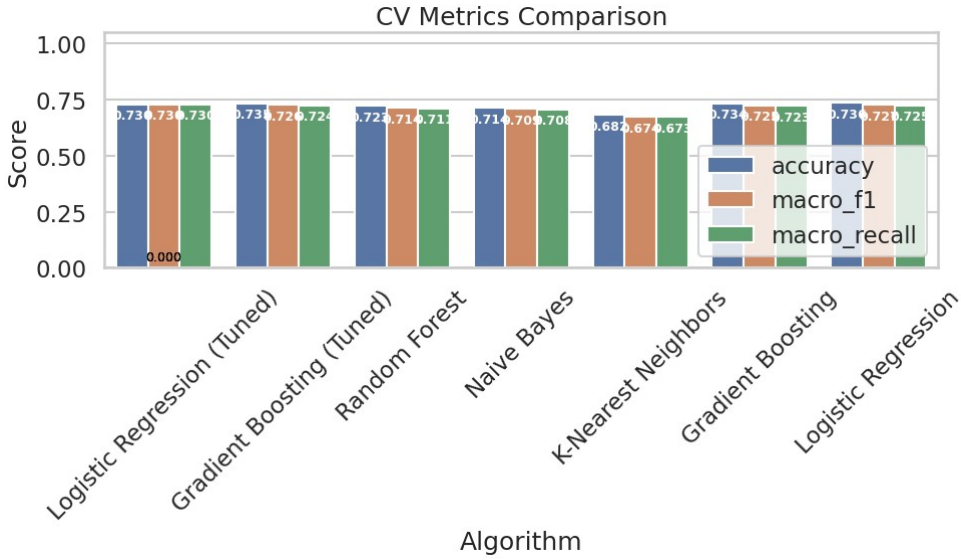
```
    pipeline_gb = Pipeline([
    ('scaler', StandardScaler()),
    ('model', GradientBoostingClassifier(random_state=42))
])

param_grid = {
    'model__n_estimators': [100, 200, 300],
    'model__learning_rate': [0.01, 0.05, 0.1],
    'model__max_depth': [3, 5, 7],
    'model__subsample': [0.7, 1.0]
}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

scoring = make_scorer(recall_score, average='macro')

grid_search = GridSearchCV(
    estimator=pipeline_gb,
    param_grid=param_grid,
    scoring=scoring,
    cv=cv,
    refit=True,
    n_jobs=-1,
    verbose=2
)

grid_search.fit(X_train, y_train)

best_gb_model = grid_search.best_estimator_

print("Best hyperparameters:", grid_search.best_params_)
print("Best CV macro F1 score:", grid_search.best_score_)
```

Kod *Gradient Boosting*-a prostor pretrage je bio složeniji – podešavani su n_estimators (broj stabala), learning_rate (korak učenja, tj. koliko svako novo stablo doprinosi finalnom modelu), max_depth (maksimalna dubina svakog stabla) i subsample (udeo podataka koji se koristi za treniranje svakog stabla). Ovi parametri međusobno interaguju – na primer, manji learning_rate tipično zahteva veći broj stabala da bi se postigli dobri rezultati.

```
y_pred_test, y_proba_test = train_and_predict(best_gb_model, X_train, y_train, X_test)
classes = np.unique(y_train)

plot_confusion_matrix(y_test, y_pred_test, classes, "Confusion Matrix – Gradient Boosting
(Tuned)")
plot_roc(y_test, y_proba_test, classes, "ROC Curve – Gradient Boosting (Tuned)")

cv_results = evaluate_cv(best_gb_model, X_train, y_train, cv)

gb_result = ModelResult(
    name="Gradient Boosting (Tuned)",
    cv_metrics=cv_results,
    y_pred_test=y_pred_test,
    y_proba_test=y_proba_test
)
```

```
tuned_parameters_results_list.append(gb_result)
```

Cilj ovog koraka je bio smanjiti *overfitting* i izvući maksimalne performanse iz svakog modela pre finalne evaluacije.



Rezultati koji su dobijeni se malo razlikuju od prvobitnih rezultata po *default* vrednostima. Ovaj korak u nekim slučajevima može značajno optimizovati predikciju u zavisnosti od samih podataka i ciljeva.

4. Zaključak

U ovom radu istražena je primena metoda mašinskog učenja za predikciju kardiovaskularnih bolesti kao binarni klasifikacioni problem. Korišćenjem sintetičkog dataseta od 15.000 zapisa, koji obuhvata demografske podatke, vitalne znakove, laboratorijske nalaze i faktore životnog stila, implementirano je i evaluirano pet algoritama: *Logistic regresion*, *Naive Bayes*, *K-nearest neighbours*, *Random Forest* i *Gradient Boosting*. Rezultati *cross* validacije pokazali su da su *Gradient Boosting* i *Logistic regresion* postigli najbolje performanse, sa accuracy skorom oko 0.735, dok je *KNN* pokazao najslabije rezultate. Primenom optimizacije hiperparametara dodatno su poboljšane performanse dva najbolja modela.

Ovi rezultati potvrđuju da metode mašinskog učenja predstavljaju vredan alat u predikciji kardiovaskularnog rizika, sposoban da prepozna složene obrasce u medicinskim podacima koje tradicionalni klinički alati poput SCORE sistema ne mogu lako da detektuju i nemaju statičan karakter.

5. Literatura

1. Institut za javno zdravlje Srbije „Dr Milan Jovanović-Batut“. *Kardiovaskularne bolesti u Srbiji*, https://www.batut.org.rs/download/novosti/Pres%20materijal%20-%20Kardiovaskularne%20bolesti%20u%20Srbiji.pdf

2. PubMed https://pubmed.ncbi.nlm.nih.gov/34140109/

3. *Framingham heart study* https://www.framinghamheartstudy.org/fhs-risk-functions/cardiovascular-disease-10-year-risk/

4. *NumPy* bibliotka https://numpy.org/

5. *scikit-learn* bibliotka https://scikit-learn.org

6. *Machine Learning with Python Cookbook,* Kyle Gallatin, Chris Albon, O'Reilly Media*, ISBN (2nd Ed.): 978-1098135720*